ProLecture

Бенчмарк проактивности, упирающейся в рассуждение

Модель смотрит лекцию в реальном времени и сама решает,
когда ответить и **что** ответить.

Материал — лекции 3Blue1Brown. Статус: пилот, 10 сэмплов, конвейер разметки собран.

Ниша

проактивность там, где ответ требует вывода, а не считывания

Работа	Что меряет	Чего не хватает
OVO-Bench	backward / real-time / forward	MCQ, нет аудио, перцепция
ProactiveVideoQA	open-ended, метрика PAUC	простые вопросы «сообщи, когда X»
OmniPro	6 способностей $\times$ 9 подзадач	84% аудио, но vision-only не отстают

- В лекции почти **нет перцептивной сложности**: чистый рендер, крупные формулы, чёткая речь.
- Зато есть логическая структура: посылка $\rightarrow$ вывод $\rightarrow$ следствие.
- Значит, ответить вовремя можно, только **поняв содержание** — а не распознав объект.

Почему 3Blue1Brown

уникальный актив: у нас есть исходный код картинки

Для **140** видео известны одновременно ссылка на YouTube и существующий путь к manim-исходнику сцены.

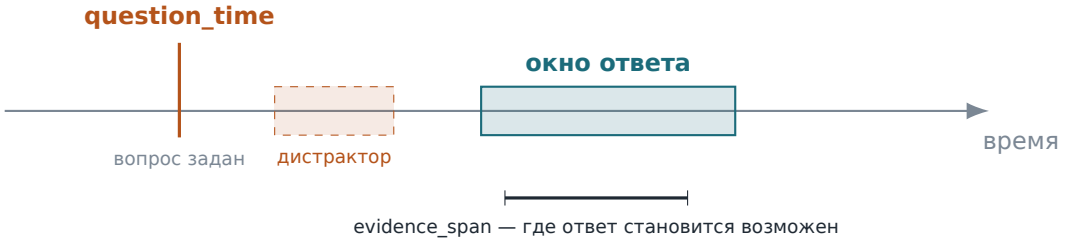
Ни у одного из существующих бенчмарков исходников видеоряда нет.

- точная ground-truth того, что на экране (строки Tex с LaTeX);
- семантические границы: имена сцен, порядок анимаций;
- **ре-рендер с внедрённой ошибкой** — контент, которого гарантированно не было в pretraining.

Связь «урок ↔ ролик ↔ код» существует только во frontmatter сайта: в самом репозитории `3b1b/videos` нет ни одной ссылки на YouTube.

Анатомия сэмпла

вопрос — в одну секунду, ответ — в окне, доказательство — отдельно



- Ось `temporal_rel` (`pre` / `at` / `post`) **проверяется валидатором**: он пересчитывает её из `question_time` и `evidence_span` и отвергает несовпадение.
- `distractor_windows` — моменты, где ответ выглядит уместным, но преждевременен.
- Подтверждение эталона живёт отдельно, в `provenance`: оно может лежать позже окна и даже вне ролика.

Что ещё лежит в сэмпле

чем судят ответ и что в разметке измерено, а не заявлено

В каждом окне

`response` — эталонный ответ

`rubric` — по чему судья ставит балл

`trigger_modality` — visual / speech / sound

`depends_on` — ссылка на предыдущее окно

Рубрика — то, что заменяет MCQ. Из `q01`:

> «названо число 2.5»
> «если расчёт показан — он взвешенный, с 1/2, 1/4, 1/8, 1/8»
> **«НЕ засчитывать 1.75 — это исходный код, а не переставленный»**

`validation` — результаты замеров

По флагу на зонд, каждый заполняется ответом модели, а не мнением:

`solvable` — решается ли вообще

`blind_solved` — решается без видео

`textonly_solved` — хватает транскрипта

`visiononly_solved` — хватает кадров

`frozenframe_solved` — хватает одного кадра

Отсюда выводится `audio_dependency`; мнение разметчика живёт отдельно, в `audio_dependency_claim`, и на выводы не влияет.

`status`: `draft` → `validated` / `rejected`.

Три оси таксономии

каждый сэмпл — точка в кубе (акт, отношение, уровень рассуждения)

A. Тип акта

`alert` — сигнал
`monitor` — состояние
`ground` — локализация
`narrate` — описание
`predict` — предсказание
`verify` — верификация
`counterfact` — контрфактика

B. Отношение к триггеру

`pre` — доказательство после вопроса: **умение ждать**

`at` — здесь и сейчас

`post` — ответ по памяти

Один вопрос в двух режимах даёт контролируемую пару.

C. Уровень рассуждения

`L1` перцепция — считывается

`L2` интеграция — связать два момента

`L3` вывод — провести рассуждение, которого в видео нет

Критерий L3: blind-бейзлайн не решает.

Ось C — то, чего нет у OmniPro: там модели не различаются именно по ней.

Принципы, из которых вырос дизайн

каждый пункт закрывает провал, на котором погорела одна из разобранных работ

- **Метки ставятся замером, а не декларацией.** `audio_dependency` выводится из ответов зондов модальностей; мнение разметчика живёт в отдельном поле и на выводы не влияет.
- **Генератор ≠ судья ≠ участник** — правило на этап замеров. Сейчас пропозер, ревьюеры и координатор — одна и та же модель, а судью и участника мы ещё не запускали. Судьёй при этом может быть чистая текстовая LLM: она читает рубрику и ответ, а видео не смотрит.
- **Открытые ответы с рубрикой, без MCQ.** Перекошенный ключ ответов ломает бенчмарк тихо.
- **Обязательные бейзлайны-пустышки:** blind, text-only, vision-only, frozen-frame, «отвечать сразу», «отвечать в конце», «молчать».
- **Никаких квот** вида «ровно N сэмплов на задачу»: сколько прошло валидацию, столько и есть.
- **Контаминация измеряется, а не отрицается:** пилот на видео 2026 года, контрольное видео 2017 года — чтобы разрыв blind-бейзлайна увидеть числом.

Как это работает на живых сэмплах

одна и та же проверка вслепую, противоположные исходы

q01 — проверку прошёл

«Как только впервые появится таблица кодовых слов разной длины с вероятностями — посчитай, каким станет среднее число бит, если поменять местами „вверх“ и „влево“».

Вопрос на 250-й секунде, таблица — на 314-й; ответа 2.5 бита в видео нет.

Полу-слепой ревьюер назвал 2.5 — и засчитан не был: его ответ начинался с «точного числа назвать не могу», дальше условная догадка и ветка с 2.0. **Угадана постановка, а не найден ответ.**

q08 — отвергнут

«Скажи до того, как автор расскажет сам: к какому источнику вероятностей букв обратился Шеннон и почему у него нет проблемы таблицы частот».

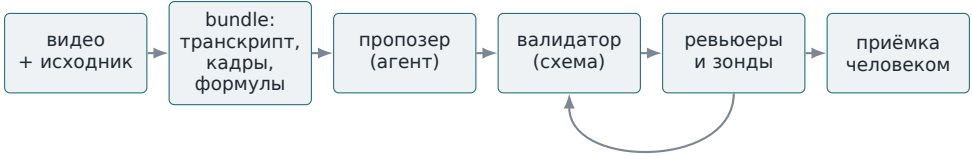
До просмотра ревьюер выдал **обе** половины ответа: живой носитель языка и рост числа контекстов как 27^n. Три пункта рубрики из трёх, без единого допущения о кадре.

`blind_solved = true`, `status: rejected`; заодно уровень понижен `L3` → `L1`.

Правило: опора на общеизвестный факт — контаминация, догадка из постановки — нет.

Конвейер

от ролика до сэмпла, прошедшего ревью



- **Валидатор** играет роль тестов: непересекающиеся окна, ссылки только назад, дистрактор не совпадает с триггером, `temporal_rel` сходится с доказательством. Возвращает код 1 при любой ошибке.
- **Ревьюеры и зонды** — следующие два слайда: три роли проверяют разметку с разных сторон, четыре зонда заполняют флаги `validation` замером.
- **Круг повторяется** до тех пор, пока все вердикты не станут `PASS`; на повторный круг уходят свежие субагенты, не видевшие прошлых обсуждений.

Три ревьюера, и каждый не знает о других

запускаются параллельно; файлы не правит никто из них

- **Полу-слепой.** Видит материал до `t_end`, но не знает `t_start` — и сам называет момент, когда ответ стал возможен. Проверяющий, который знает ответ, «узнаёт» его на полукадре и раз за разом двигает левую границу окна влево; этого знания у него нет.

  Отдельно записывает `prior_answer` — свой ответ **до** просмотра, дословно. Из него координатор и выводит `blind_solved`.
- **Ревьюер сэмпла.** Всё, что относится к вопросу как целому: формулировка, рубрика, приманки, эталонный ответ, ссылки между окнами. Границы окон — не его работа.
- **Пооконный ревьюер, по одному на окно.** Роль отделена не для симметрии: у одного ревьюера на вопросе с девятью окнами бюджета кадров хватало на первое, а дальше он додумывал.

Каждый возвращает структурированное ревью по контракту (`schema/review.py`), а правки применяет координатор — так видно, кто что нашёл, и повторный круг уходит свежим субагентам.

Четыре зонда: метка получается замером

каждому дают ровно один канал и смотрят, ответит ли

Зонд	Материал	Кадров	Заполняет
`text`	транскрипт до `t_end`	0	`textonly_solved`
`frame`	один кадр в середине окна	1	`frozenframe_solved`
`vision`	кадры до `t_end`	до 40	`visiononly_solved`
`full`	транскрипт и кадры	до 40	`solvable`

`solvable = textonly OR visiononly` OR ответ `full`; `solvable = false` — `CRITICAL_FAIL`, вопрос не решается и по полному материалу.

`visiononly_solved` вместе с `solvable` дают **`audio_dependency`** — руками эта метка не ставится.

`frozenframe_solved = true` при уровне L2/L3 значит, что уровень завышен.

Изоляция зонда держится на трёх вещах: промпт просит не лезть в чужой канал, скрипт не даёт смешать материал двух режимов в одной папке, а сам зонд обязан признаться в поле `violations`.

Инструменты, которых не было

два места, где готовое не подошло

Свой детектор смены сцен

Готовые детекторы сравнивают кадр с предыдущим — а типовой переход manim это плавный переезд за 2–4 секунды, где соседние кадры почти не отличаются.

Считаем **накопленный дрейф** от последнего снятого кадра и снимаем, когда анимация улеглась.

Один порог на домен: **13.5** / **12.9** / **10.1** кадра на минуту на роликах 2026 и 2017 годов.

На контрольном участке: **18** состояний из 18 против **10** у равномерной сетки.

Плотное описание видеоряда

Документ вроде субтитров, только вместо реплик — что происходит на экране. Разметчик читает текст вместо просмотра десятков кадров.

Два кадра в секунду, окна по 5 секунд с перекрытием, строгий JSON по схеме (guided decoding на стороне сервера).

Пилот: **3878** кадров, **485** фрагментов, **20** минут на двух A100, **1939** секунд из 1939 покрыты событиями.

Субтитры описывающей модели не показывают: она описывает только кадры. Иначе описание нельзя было бы отдать зонду vision-only вместо кадров.

Где мы сейчас

Сделано

- конвейер данных: маппинг → скачивание → bundle;
- схема сэмпла и валидатор семантики;
- пилот: **10** сэмплов, **24** триггера и **12** дистракторов, ошибок валидации ноль;
- промпты пропозера, трёх ревьюеров и четырёх зондов;
- детектор сцен и описание видеоряда.

Дальше

- прогнать бейзлайны-пустышки и увидеть, что они проваливаются;
- замерить разрыв blind на контрольном видео 2017 года;
- harness потоковой выдачи и LLM-судья по рубрике;
- расширение разметки: доля L3 должна вырасти;
- опционально — ре-рендер сцен с внедрённой ошибкой.

Открытые допущения записаны отдельно и в Limitations, а не заметены: утечка объяснения дистрактора, высокая сложность пилота, малый объём.

Метрика

спроектирована, но ещё не реализована: задел на фазу замеров

PAUC. На каждом окне строится ломаная «время $\rightarrow$ качество накопленных ответов», качество $s \in \{0, 1, 2\}$ ставит LLM-судья по рубрике.

Стартовая точка $(t_{start}, 0.5)$: молчание лучше уверенного неверного ответа.

Следствие, которое отчитываем явно: стратегия «молчать всегда» получает **0.25** от максимума. Это пол шкалы, а не ноль.

Обязательно рядом:

- **False-Alarm Rate** — ответы вне окон
- **Trap rate** — попадания в дистракторы, отдельно от FAR
- **Latency** — от t_{start}, распределением
- **Silence rate**, **Trigger Recall**
- **Responses-per-sample** — иначе спам-стратегия невидима

Агрегация по сэмплам, а не по окнам: иначе итог — заложник одного самого простого сэмпла с девятью триггерами.